

Admin Panel — Pages

Every page in the dashboard with purpose, data requirements, and full TypeScript component code.

1. Login Page

File: app/(auth)/login/page.tsx

```
"use client";

import { useState } from "react";
import { signIn } from "next-auth/react";
import { useRouter, useSearchParams } from "next/navigation";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Loader2, ShieldCheck } from "lucide-react";

const loginSchema = z.object({
  email: z.string().email("Valid email required"),
  password: z.string().min(6, "Password must be at least 6 characters"),
});

type LoginForm = z.infer<typeof loginSchema>;

export default function LoginPage() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const [error, setError] = useState<string | null>(null);

  const errorParam = searchParams.get("error");

  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm<LoginForm>({ resolver: zodResolver(loginSchema) });
```

```

async function onSubmit(data: LoginForm) {
  setError(null);
  const result = await signIn("credentials", {
    email: data.email,
    password: data.password,
    redirect: false,
  });

  if (result?.error) {
    setError("Invalid email or password. Please try again.");
  } else {
    router.push("/");
    router.refresh();
  }
}

return (
  <div className="min-h-screen flex items-center justify-center bg-gray-50">
    <div className="w-full max-w-md px-4">
      <div className="flex flex-col items-center mb-8">
        <div className="flex items-center justify-center h-14 w-14 rounded-2xl bg-gray-900 mb-4">
          <ShieldCheck className="h-7 w-7 text-white" />
        </div>
        <h1 className="text-2xl font-bold text-gray-900">b-secure Admin</h1>
        <p className="text-sm text-gray-500 mt-1">Internal operations dashboard</p>
      </div>

      <Card className="shadow-sm border border-gray-200">
        <CardHeader>
          <CardTitle className="text-lg">Sign in to your account</CardTitle>
        </CardHeader>
        <CardContent>
          {(error || errorParam) && (
            <div className="mb-4 rounded-lg bg-red-50 border border-red-200 px-4 py-3 text-sm text-red-700">
              {error ||
                (errorParam === "session_expired"
                  ? "Your session expired. Please sign in again."
                  : errorParam === "unauthorized"
                  ? "You do not have admin access."
                  : "An error occurred. Please try again.")}
            </div>
          )}
        </CardContent>
      </Card>

      <form onSubmit={handleSubmit(onSubmit)}
        className="space-y-4">
        <div className="space-y-1.5">

```

```

        <Label htmlFor="email">Email</Label>
        <Input
            id="email"
            type="email"
            placeholder="admin@bsecure.co.za"
            autoComplete="email"
            {...register("email")}
        />
        {errors.email && (
            <p className="text-xs text-red-600">{errors.email.message}</p>
        )}
    </div>

    <div className="space-y-1.5">
        <Label htmlFor="password">Password</Label>
        <Input
            id="password"
            type="password"
            autoComplete="current-password"
            {...register("password")}
        />
        {errors.password && (
            <p className="text-xs text-red-600">{errors.password.message}</p>
        )}
    </div>

    <Button
        type="submit"
        className="w-full bg-gray-900 hover:bg-gray-800"
        disabled={isSubmitting}
    >
        {isSubmitting ? (
            <>
                <Loader2 className="mr-2 h-4 w-4 animate-spin" />
                Signing in...
            </>
        ) : (
            "Sign in"
        )}
    </Button>
</form>
</CardContent>
</Card>
</div>
</div>
);
}

```

2. Dashboard Overview

File: app/(dashboard)/page.tsx

```
"use client";

import { useQuery } from "@tanstack/react-query";
import { dashboardService } from "@services/dashboardService";
import { KPICard } from "@components/shared/KPICard";
import { BookingsLineChart } from "@components/charts/BookingsLineChart";
import { DataTable } from "@components/tables/DataTable";
import { StatusBadge } from "@components/shared/StatusBadge";
import { PageHeader } from "@components/shared/PageHeader";
import { useAdminWebSocket } from "@hooks/useAdminWebSocket";
import { useDashboardStore } from "@stores/dashboardStore";
import { formatCurrency, formatDateTime } from "@lib/utils";
import { CalendarCheck, ShieldCheck, AlertTriangle, DollarSign } from "lucide-react";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import type { ColumnDef } from "@tanstack/react-table";
import type { SOSEvent } from "@types/admin";

const sosColumns: ColumnDef<SOSEvent>[] = [
  { accessorKey: "id", header: "ID", cell: ({ getValue }) => `#${getValue()}` },
  { accessorKey: "user_name", header: "User" },
  { accessorKey: "guard_name", header: "Guard" },
  {
    accessorKey: "status",
    header: "Status",
    cell: ({ getValue }) => <StatusBadge
status={getValue<string>()} />,
  },
  {
    accessorKey: "created_at",
    header: "Time",
    cell: ({ getValue }) => formatDateTime(getValue<string>()),
  },
];

export default function DashboardPage() {
  // Connect WebSocket
  useAdminWebSocket();

  const { activeBookings, onlineGuards, sosCount, revenueToday } =
    useDashboardStore();

  const { data: metrics, isLoading: metricsLoading } = useQuery({
    queryKey: ["dashboard", "metrics"],
    queryFn: dashboardService.getOverviewMetrics,
    refetchInterval: 30_000,
  });
```

```

});

const { data: recentSOS, isLoading: sosLoading } = useQuery({
  queryKey: ["dashboard", "recent-sos"],
  queryFn: () => dashboardService.getRecentSOS(5),
  refetchInterval: 30_000,
});

const { data: hourlyData } = useQuery({
  queryKey: ["dashboard", "hourly-bookings"],
  queryFn: dashboardService.getHourlyBookings,
  refetchInterval: 30_000,
});

const kpiCards = [
  {
    title: "Active Bookings",
    value: activeBookings ?? metrics?.active_bookings ?? 0,
    icon: CalendarCheck,
    trend: metrics?.bookings_trend,
    color: "blue" as const,
  },
  {
    title: "Online Guards",
    value: onlineGuards ?? metrics?.online_guards ?? 0,
    icon: ShieldCheck,
    trend: metrics?.guards_trend,
    color: "green" as const,
  },
  {
    title: "SOS Events",
    value: sosCount ?? metrics?.active_sos ?? 0,
    icon: AlertTriangle,
    trend: metrics?.sos_trend,
    color: "red" as const,
    live: true,
  },
  {
    title: "Revenue Today",
    value: formatCurrency(revenueToday ??
metrics?.revenue_today ?? 0),
    icon: DollarSign,
    trend: metrics?.revenue_trend,
    color: "purple" as const,
  },
];

return (
  <div className="space-y-6">
    <PageHeader
      title="Dashboard Overview"
      description="Real-time snapshot of platform activity"
    />

```

```

        {/* KPI Cards */}
        <div className="grid grid-cols-1 gap-4 sm:grid-cols-2
xl:grid-cols-4">
            {kpiCards.map((card) => (
                <KPICard key={card.title} {...card}
loading={metricsLoading} />
            ))}
        </div>

        {/* Charts + Map row */}
        <div className="grid grid-cols-1 gap-6 xl:grid-cols-2">
            {/* Hourly bookings chart */}
            <Card>
                <CardHeader>
                    <CardTitle className="text-base">Hourly Bookings</
CardTitle>
                </CardHeader>
                <CardContent>
                    <BookingsLineChart data={hourlyData ?? []} />
                </CardContent>
            </Card>

            {/* Live map */}
            <Card>
                <CardHeader>
                    <CardTitle className="text-base">Active Bookings Map</
CardTitle>
                </CardHeader>
                <CardContent className="p-0 rounded-b-lg overflow-
hidden">
                    <iframe
                        title="Active bookings map"
                        width="100%"
                        height="280"
                        className="border-0"
                        loading="lazy"
                        src={`https://www.google.com/maps/embed/v1/view?
key=${process.env.NEXT_PUBLIC_GOOGLE_MAPS_KEY}
&center=-26.2041,28.0473&zoom=11`}
                    />
                </CardContent>
            </Card>
        </div>

        {/* Recent SOS */}
        <Card>
            <CardHeader>
                <CardTitle className="text-base">Recent SOS Events</
CardTitle>
            </CardHeader>
            <CardContent>
                <DataTable
                    columns={sosColumns}
                    data={recentSOS ?? []}

```

```

        loading={sosLoading}
        pageSize={5}
        disablePagination
      />
    </CardContent>
  </Card>
</div>
);
}

```

3. Guards List

File: app/(dashboard)/guards/page.tsx

```

"use client";

import { useState } from "react";
import { useQuery } from "@tanstack/react-query";
import { guardService } from "@services/guardService";
import { DataTable } from "@components/tables/DataTable";
import { PageHeader } from "@components/shared/PageHeader";
import { StatusBadge } from "@components/shared/StatusBadge";
import { ExportButton } from "@components/shared/ExportButton";
import { Input } from "@components/ui/input";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@components/ui/select";
import { Avatar, AvatarFallback, AvatarImage } from "@components/ui/avatar";
import { Button } from "@components/ui/button";
import { Badge } from "@components/ui/badge";
import { useRouter } from "next/navigation";
import { formatDate } from "@lib/utils";
import { Eye, Search } from "lucide-react";
import type { ColumnDef, PaginationState, SortingState } from "@tanstack/react-table";
import type { Guard } from "@types/admin";

export default function GuardsPage() {
  const router = useRouter();
  const [search, setSearch] = useState("");
  const [tier, setTier] = useState<string>("all");
  const [verificationStatus, setVerificationStatus] =
    useState<string>("all");
  const [isOnline, setIsOnline] = useState<string>("all");
  const [pagination, setPagination] = useState<PaginationState>({
    pageIndex: 0,

```

```

    pageSize: 20,
  });
  const [sorting, setSorting] = useState<SortingState>([]);

  const { data, isLoading } = useQuery({
    queryKey: ["guards", { search, tier, verificationStatus,
isOnline, pagination, sorting }],
    queryFn: () =>
      guardService.listGuards({
        search: search || undefined,
        tier: tier !== "all" ? tier : undefined,
        verification_status: verificationStatus !== "all" ?
verificationStatus : undefined,
        is_online: isOnline !== "all" ? isOnline === "true" :
undefined,
        page: pagination.pageIndex + 1,
        page_size: pagination.pageSize,
        ordering: sorting[0]
          ? `${sorting[0].desc ? "-" : ""}${sorting[0].id}`
          : undefined,
      }),
    placeholderData: (prev) => prev,
  });

  const columns: ColumnDef<Guard>[] = [
    {
      id: "photo",
      header: "",
      cell: ({ row }) => (
        <Avatar className="h-8 w-8">
          <AvatarImage src={row.original.photo_url}
alt={row.original.name} />
          <AvatarFallback className="text-xs">
            {row.original.name.slice(0, 2).toUpperCase()}
          </AvatarFallback>
        </Avatar>
      ),
      enableSorting: false,
    },
    { accessorKey: "name", header: "Name" },
    {
      accessorKey: "tier",
      header: "Tier",
      cell: ({ getValue }) => (
        <Badge variant="outline" className="capitalize">
          {getValue<string>()}
        </Badge>
      ),
    },
    {
      accessorKey: "is_online",
      header: "Online",
      cell: ({ getValue }) => (
        <span

```



```

        className={`inline-flex items-center gap-1.5 text-xs
font-medium ${
    getValue<boolean>() ? "text-green-600" : "text-
gray-400"
  }`}
      >
        <span
          className={`h-1.5 w-1.5 rounded-full ${
            getValue<boolean>() ? "bg-green-500" : "bg-gray-300"
          }`}
        />
        {getValue<boolean>() ? "Online" : "Offline"}
      </span>
    ),
  },
  {
    accessorKey: "rating",
    header: "Rating",
    cell: ({ getValue }) => `${(getValue<number>() ??
0).toFixed(1)} ★`,
  },
  { accessorKey: "total_bookings", header: "Bookings" },
  {
    accessorKey: "verification_status",
    header: "Verification",
    cell: ({ getValue }) => <StatusBadge
status={getValue<string>()} />,
  },
  {
    accessorKey: "created_at",
    header: "Joined",
    cell: ({ getValue }) => formatDate(getValue<string>()),
  },
  {
    id: "actions",
    header: "",
    cell: ({ row }) => (
      <Button
        variant="ghost"
        size="sm"
        onClick={() => router.push(`/guards/${row.original.id}
`)}
      >
        <Eye className="h-4 w-4 mr-1" />
        View
      </Button>
    ),
  },
];

return (
  <div className="space-y-6">
    <PageHeader
      title="Guards"

```

```

description="Manage all registered guards"
action={
  <ExportButton
    endpoint="/admin/guards/export/"
    filename="guards.csv"
  />
}
/>

{/* Filters */}
<div className="flex flex-wrap gap-3">
  <div className="relative flex-1 min-w-[200px]">
    <Search className="absolute left-3 top-1/2 -translate-
y-1/2 h-4 w-4 text-gray-400" />
    <Input
      placeholder="Search name or phone..."
      value={search}
      onChange={(e) => {
        setSearch(e.target.value);
        setPagination((p) => ({ ...p, pageIndex: 0 }));
      }}
      className="pl-9"
    />
  </div>

  <Select value={tier} onValueChange={setTier}>
    <SelectTrigger className="w-36">
      <SelectValue placeholder="Tier" />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="all">All Tiers</SelectItem>
      <SelectItem value="bronze">Bronze</SelectItem>
      <SelectItem value="silver">Silver</SelectItem>
      <SelectItem value="gold">Gold</SelectItem>
      <SelectItem value="platinum">Platinum</SelectItem>
    </SelectContent>
  </Select>

  <Select value={verificationStatus}
onValueChange={setVerificationStatus}>
    <SelectTrigger className="w-44">
      <SelectValue placeholder="Verification" />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="all">All Statuses</SelectItem>
      <SelectItem value="pending">Pending</SelectItem>
      <SelectItem value="approved">Approved</SelectItem>
      <SelectItem value="rejected">Rejected</SelectItem>
    </SelectContent>
  </Select>

  <Select value={isOnline} onValueChange={setIsOnline}>
    <SelectTrigger className="w-32">
      <SelectValue placeholder="Online" />
    </SelectTrigger>
  </Select>

```

```

        </SelectTrigger>
        <SelectContent>
          <SelectItem value="all">All</SelectItem>
          <SelectItem value="true">Online</SelectItem>
          <SelectItem value="false">Offline</SelectItem>
        </SelectContent>
      </Select>
    </div>

    <DataTable
      columns={columns}
      data={data?.results ?? []}
      loading={isLoading}
      totalCount={data?.count ?? 0}
      pagination={pagination}
      onPaginationChange={setPagination}
      sorting={sorting}
      onSortingChange={setSorting}
      pageSize={pagination.pageSize}
    />
  </div>
);
}

```

4. Guard Detail

File: app/(dashboard)/guards/[id]/page.tsx

```

"use client";

import { useState } from "react";
import { useParams } from "next/navigation";
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { guardService } from "@services/guardService";
import { PageHeader } from "@components/shared/PageHeader";
import { GuardVerificationCard } from "@components/shared/GuardVerificationCard";
import { BookingTimeline } from "@components/shared/BookingTimeline";
import { StatusBadge } from "@components/shared/StatusBadge";
import { ConfirmDialog } from "@components/shared/ConfirmDialog";
import { DataTable } from "@components/tables/DataTable";
import { Button } from "@components/ui/button";
import { Badge } from "@components/ui/badge";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Avatar, AvatarFallback, AvatarImage } from "@components/ui/avatar";
import {
  Select,

```

```

    SelectContent,
    SelectItem,
    SelectTrigger,
    SelectValue,
  } from "@components/ui/select";
import { formatDate, formatDateTime, formatCurrency } from "@lib/
utils";
import { CheckCircle, XCircle, Ban, Star } from "lucide-react";
import { toast } from "sonner";
import type { ColumnDef } from "@tanstack/react-table";
import type { Booking } from "@types/admin";

const bookingColumns: ColumnDef<Booking>[] = [
  { accessorKey: "id", header: "ID", cell: ({ getValue }) => `#$
{getValue()}` },
  { accessorKey: "user_name", header: "User" },
  {
    accessorKey: "status",
    header: "Status",
    cell: ({ getValue }) => <StatusBadge
status={getValue<string>()} />,
  },
  {
    accessorKey: "amount",
    header: "Amount",
    cell: ({ getValue }) => formatCurrency(getValue<number>()),
  },
  {
    accessorKey: "created_at",
    header: "Date",
    cell: ({ getValue }) => formatDate(getValue<string>()),
  },
];

export default function GuardDetailPage() {
  const { id } = useParams<{ id: string }>();
  const queryClient = useQueryClient();
  const [suspendOpen, setSuspendOpen] = useState(false);
  const [approveOpen, setApproveOpen] = useState(false);
  const [selectedTier, setSelectedTier] = useState("");

  const { data: guard, isLoading } = useQuery({
    queryKey: ["guard", id],
    queryFn: () => guardService.getGuard(id),
  });

  const { data: bookings, isLoading: bookingsLoading } =
useQuery({
  queryKey: ["guard", id, "bookings"],
  queryFn: () => guardService.getGuardBookings(id),
});

  const approveMutation = useMutation({
    mutationFn: () => guardService.approveGuard(id),
  });

```

```

      onSuccess: () => {
        toast.success("Guard approved successfully");
        queryClient.invalidateQueries({ queryKey: ["guard", id] });
        setApproveOpen(false);
      },
      onError: (err: Error) => toast.error(err.message),
    });

const suspendMutation = useMutation({
  mutationFn: () => guardService.suspendGuard(id),
  onSuccess: () => {
    toast.success("Guard suspended");
    queryClient.invalidateQueries({ queryKey: ["guard", id] });
    setSuspendOpen(false);
  },
  onError: (err: Error) => toast.error(err.message),
});

const changeTierMutation = useMutation({
  mutationFn: (tier: string) => guardService.changeGuardTier(id,
tier),
  onSuccess: () => {
    toast.success("Tier updated");
    queryClient.invalidateQueries({ queryKey: ["guard", id] });
  },
  onError: (err: Error) => toast.error(err.message),
});

if (isLoading || !guard) {
  return (
    <div className="flex items-center justify-center h-64">
      <div className="animate-spin h-8 w-8 rounded-full border-2
border-gray-300 border-t-gray-900" />
    </div>
  );
}

return (
  <div className="space-y-6">
    <PageHeader
      title={guard.name}
      description={`Guard ID: ${guard.id}`}
      action={
        <div className="flex gap-2">
          {guard.verification_status === "pending" && (
            <Button
              size="sm"
              className="bg-green-600 hover:bg-green-700"
              onClick={() => setApproveOpen(true)}
            >
              <CheckCircle className="mr-2 h-4 w-4" />
              Approve Guard
            </Button>
          )}
        </div>
      }
    />
  )

```

```

        <Button
          variant="destructive"
          size="sm"
          onClick={() => setSuspendOpen(true)}
        >
          <Ban className="mr-2 h-4 w-4" />
          Suspend
        </Button>
      </div>
    }
  />

  { /* Profile */}
  <div className="grid grid-cols-1 gap-6 xl:grid-cols-3">
    <Card className="xl:col-span-1">
      <CardContent className="pt-6">
        <div className="flex flex-col items-center gap-4 text-
center">
          <Avatar className="h-24 w-24">
            <AvatarImage src={guard.photo_url}
alt={guard.name} />
            <AvatarFallback className="text-2xl">
              {guard.name.slice(0, 2).toUpperCase()}
            </AvatarFallback>
          </Avatar>
          <div>
            <h2 className="text-xl font-semibold">{guard.name}
</h2>
            <p className="text-sm text-gray-500">{guard.phone}
</p>
            <p className="text-sm text-gray-500">{guard.email}
</p>
          </div>
          <div className="flex items-center gap-2">
            <StatusBadge status={guard.verification_status} />
            <Badge variant="outline" className="capitalize">
              {guard.tier}
            </Badge>
          </div>
          <div className="flex items-center gap-1 text-
amber-500">
            <Star className="h-4 w-4 fill-current" />
            <span className="font-
semibold">{guard.rating.toFixed(1)}</span>
            <span className="text-gray-400 text-sm">
              ({guard.total_ratings} reviews)
            </span>
          </div>
        </div>
      </div>

      { /* Change Tier */}
      <div className="mt-6 space-y-2">
        <label className="text-sm font-medium">Change Tier</
label>

```

```

<div className="flex gap-2">
  <Select
    value={selectedTier || guard.tier}
    onValueChange={setSelectedTier}
  >
    <SelectTrigger>
      <SelectValue />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="bronze">Bronze</SelectItem>
      <SelectItem value="silver">Silver</SelectItem>
      <SelectItem value="gold">Gold</SelectItem>
      <SelectItem value="platinum">Platinum</
SelectItem>
      </SelectContent>
    </Select>
    <Button
      size="sm"
      disabled={!selectedTier || selectedTier ===
guard.tier}
      onClick={() =>
changeTierMutation.mutate(selectedTier)}
    >
      Save
    </Button>
  </div>
</div>
</CardContent>
</Card>

{/* Stats */}
<Card className="xl:col-span-2">
  <CardHeader>
    <CardTitle className="text-base">Guard Stats</
CardTitle>
  </CardHeader>
  <CardContent>
    <dl className="grid grid-cols-2 gap-4 sm:grid-cols-3">
      {[
        { label: "Total Bookings", value:
guard.total_bookings },
        { label: "Completion Rate", value: ` $
{guard.completion_rate}%` },
        {
          label: "Total Earned",
          value: formatCurrency(guard.total_earned),
        },
        { label: "Joined", value:
formatDate(guard.created_at) },
        { label: "Last Active", value:
formatDateTime(guard.last_active) },
        { label: "ID Number", value: guard.id_number ||
"-" },
      ]}.map(({ label, value }) => (

```

```

        <div key={label}>
          <dt className="text-xs text-gray-500">{label}</
dt>
          <dd className="mt-0.5 text-sm font-semibold
text-gray-900">
            {value}
          </dd>
        </div>
      )))
    </dl>
  </CardContent>
</Card>
</div>

{ /* Documents */}
{guard.documents && guard.documents.length > 0 && (
  <div className="space-y-3">
    <h2 className="text-base font-semibold">Verification
Documents</h2>
    <div className="grid grid-cols-1 gap-4 sm:grid-cols-2
xl:grid-cols-3">
      {guard.documents.map((doc) => (
        <GuardVerificationCard
          key={doc.id}
          document={doc}
          guardId={id}
          onUpdate={() =>
            queryClient.invalidateQueries({ queryKey:
["guard", id] })
          }
        </>
      ))}
    </div>
  </div>
)}

{ /* Booking History */}
<Card>
  <CardHeader>
    <CardTitle className="text-base">Booking History</
CardTitle>
  </CardHeader>
  <CardContent>
    <DataTable
      columns={bookingColumns}
      data={bookings?.results ?? []}
      loading={bookingsLoading}
      totalCount={bookings?.count ?? 0}
      pageSize={10}
    </>
  </CardContent>
</Card>

{ /* Audit Log */}

```



```

    {guard.audit_log && guard.audit_log.length > 0 && (
      <Card>
        <CardHeader>
          <CardTitle className="text-base">Audit Log</CardTitle>
        </CardHeader>
        <CardContent>
          <BookingTimeline events={guard.audit_log} />
        </CardContent>
      </Card>
    )}

    {/* Confirm Dialogs */}
    <ConfirmDialog
      open={approveOpen}
      onOpenChange={setApproveOpen}
      title="Approve Guard"
      description={`Are you sure you want to approve $
{guard.name}? They will be able to accept bookings.`}
      confirmLabel="Approve"
      variant="default"
      onConfirm={() => approveMutation.mutate()}
      loading={approveMutation.isPending}
    />

    <ConfirmDialog
      open={suspendOpen}
      onOpenChange={setSuspendOpen}
      title="Suspend Guard"
      description={`Suspending ${guard.name} will prevent them
from accepting new bookings. Continue?`}
      confirmLabel="Suspend"
      variant="destructive"
      onConfirm={() => suspendMutation.mutate()}
      loading={suspendMutation.isPending}
    />
  </div>
);
}

```

5. Users List

File: app/(dashboard)/users/page.tsx

```

"use client";

import { useState } from "react";
import { useQuery } from "@tanstack/react-query";
import { userService } from "@services/userService";
import { DataTable } from "@components/tables/DataTable";
import { PageHeader } from "@components/shared/PageHeader";
import { StatusBadge } from "@components/shared/StatusBadge";

```

```

import { ExportButton } from "@components/shared/ExportButton";
import { Input } from "@components/ui/input";
import { Button } from "@components/ui/button";
import { Avatar, AvatarFallback, AvatarImage } from "@components/
ui/avatar";
import { useRouter } from "next/navigation";
import { formatDate } from "@lib/utils";
import { Eye, Search } from "lucide-react";
import type { ColumnDef, PaginationState, SortingState } from
"@tanstack/react-table";
import type { User } from "@types/admin";

export default function UsersPage() {
  const router = useRouter();
  const [search, setSearch] = useState("");
  const [pagination, setPagination] = useState<PaginationState>({
    pageIndex: 0,
    pageSize: 20,
  });
  const [sorting, setSorting] = useState<SortingState>([]);

  const { data, isLoading } = useQuery({
    queryKey: ["users", { search, pagination, sorting }],
    queryFn: () =>
      userService.listUsers({
        search: search || undefined,
        page: pagination.pageIndex + 1,
        page_size: pagination.pageSize,
        ordering: sorting[0]
          ? `${sorting[0].desc ? "-" : ""}${sorting[0].id}`
          : undefined,
      }),
    placeholderData: (prev) => prev,
  });

  const columns: ColumnDef<User>[] = [
    {
      id: "avatar",
      header: "",
      cell: ({ row }) => (
        <Avatar className="h-8 w-8">
          <AvatarImage src={row.original.photo_url} />
          <AvatarFallback className="text-xs">
            {row.original.name.slice(0, 2).toUpperCase()}
          </AvatarFallback>
        </Avatar>
      ),
      enableSorting: false,
    },
    { accessorKey: "name", header: "Full Name" },
    { accessorKey: "email", header: "Email" },
    { accessorKey: "phone", header: "Phone" },
    { accessorKey: "total_bookings", header: "Bookings" },
    {

```

```

        accessorKey: "status",
        header: "Status",
        cell: ({ getValue }) => <StatusBadge
status={getValue<string>()} />,
    },
    {
        accessorKey: "created_at",
        header: "Registered",
        cell: ({ getValue }) => formatDate(getValue<string>()),
    },
    {
        id: "actions",
        header: "",
        cell: ({ row }) => (
            <Button
                variant="ghost"
                size="sm"
                onClick={() => router.push(`/users/${row.original.id}`)}
            >
                <Eye className="h-4 w-4 mr-1" />
                View
            </Button>
        ),
    },
];

return (
    <div className="space-y-6">
        <PageHeader
            title="Users"
            description="All registered platform users"
            action={
                <ExportButton endpoint="/admin/users/export/"
filename="users.csv" />
            }
        />

        <div className="relative max-w-sm">
            <Search className="absolute left-3 top-1/2 -translate-
y-1/2 h-4 w-4 text-gray-400" />
            <Input
                placeholder="Search name, email or phone..."
                value={search}
                onChange={(e) => {
                    setSearch(e.target.value);
                    setPagination((p) => ({ ...p, pageIndex: 0 }));
                }}
                className="pl-9"
            />
        </div>

        <DataTable
            columns={columns}
            data={data?.results ?? []}

```

```

        loading={isLoading}
        totalCount={data?.count ?? 0}
        pagination={pagination}
        onPaginationChange={setPagination}
        sorting={sorting}
        onSortingChange={setSorting}
        pageSize={pagination.pageSize}
      />
    </div>
  );
}

```

6. User Detail

File: app/(dashboard)/users/[id]/page.tsx

```

"use client";

import { useParams } from "next/navigation";
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { userService } from "@services/userService";
import { PageHeader } from "@components/shared/PageHeader";
import { StatusBadge } from "@components/shared/StatusBadge";
import { ConfirmDialog } from "@components/shared/ConfirmDialog";
import { DataTable } from "@components/tables/DataTable";
import { Button } from "@components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Avatar, AvatarFallback, AvatarImage } from "@components/ui/avatar";
import { formatDate, formatCurrency } from "@lib/utils";
import { Ban } from "lucide-react";
import { toast } from "sonner";
import { useState } from "react";
import type { ColumnDef } from "@tanstack/react-table";
import type { Booking } from "@types/admin";

const bookingCols: ColumnDef<Booking>[] = [
  { accessorKey: "id", header: "ID", cell: ({ getValue }) => `#${getValue()}` },
  { accessorKey: "guard_name", header: "Guard" },
  { accessorKey: "status", header: "Status", cell: ({ getValue }) => <StatusBadge status={getValue<string>()} /> },
  { accessorKey: "amount", header: "Amount", cell: ({ getValue }) => formatCurrency(getValue<number>()) },
  { accessorKey: "created_at", header: "Date", cell: ({ getValue }) => formatDate(getValue<string>()) },
];

export default function UserDetailPage() {

```

```

const { id } = useParams<{ id: string }>();
const qc = useQueryClient();
const [suspendOpen, setSuspendOpen] = useState(false);

const { data: user, isLoading } = useQuery({
  queryKey: ["user", id],
  queryFn: () => userService.getUser(id),
});

const { data: bookings } = useQuery({
  queryKey: ["user", id, "bookings"],
  queryFn: () => userService.getUserBookings(id),
});

const suspendMutation = useMutation({
  mutationFn: () => userService.suspendUser(id),
  onSuccess: () => {
    toast.success("User suspended");
    qc.invalidateQueries({ queryKey: ["user", id] });
    setSuspendOpen(false);
  },
  onError: (err: Error) => toast.error(err.message),
});

if (isLoading || !user) return null;

return (
  <div className="space-y-6">
    <PageHeader
      title={user.name}
      description={`User ID: ${user.id}`}
      action={
        <Button variant="destructive" size="sm" onClick={() =>
setSuspendOpen(true)}>
          <Ban className="mr-2 h-4 w-4" />
          Suspend User
        </Button>
      }
    />

    <div className="grid grid-cols-1 gap-6 xl:grid-cols-3">
      <Card>
        <CardContent className="pt-6 flex flex-col items-center
gap-4 text-center">
          <Avatar className="h-20 w-20">
            <AvatarImage src={user.photo_url} />
            <AvatarFallback>{user.name.slice(0,
2).toUpperCase()}</AvatarFallback>
          </Avatar>
          <div>
            <h2 className="text-lg font-semibold">{user.name}</
h2>
            <p className="text-sm text-gray-500">{user.email}</
p>

```

```

        <p className="text-sm text-gray-500">{user.phone}</p>
    </div>
    <StatusBadge status={user.status} />
  </CardContent>
</Card>

  <Card className="xl:col-span-2">
    <CardHeader><CardTitle className="text-base">Account
    Details</CardTitle></CardHeader>
    <CardContent>
      <dl className="grid grid-cols-2 gap-4 sm:grid-cols-3">
        {[
          { label: "Total Bookings", value:
user.total_bookings },
          { label: "Total Spent", value:
formatCurrency(user.total_spent) },
          { label: "Joined", value:
formatDate(user.created_at) },
          { label: "Last Active", value:
formatDate(user.last_active) },
        ].map(({ label, value }) => (
          <div key={label}>
            <dt className="text-xs text-gray-500">{label}</dt>
            <dd className="mt-0.5 text-sm font-
semibold">{value}</dd>
          </div>
        ))}
      </dl>
    </CardContent>
  </Card>
</div>

  <Card>
    <CardHeader><CardTitle className="text-base">Booking
    History</CardTitle></CardHeader>
    <CardContent>
      <DataTable columns={bookingCols}
data={bookings?.results ?? []} totalCount={bookings?.count ?? 0}
pageSize={10} />
    </CardContent>
  </Card>

  <ConfirmDialog
    open={suspendOpen}
    onOpenChange={setSuspendOpen}
    title="Suspend User"
    description={`Suspending ${user.name} will prevent them
from making new bookings.`}
    confirmLabel="Suspend"
    variant="destructive"
    onConfirm={() => suspendMutation.mutate()}
    loading={suspendMutation.isPending}
  />

```

```

    />
  </div>
);
}

```

7. Bookings List

File: app/(dashboard)/bookings/page.tsx

```

"use client";

import { useState } from "react";
import { useQuery } from "@tanstack/react-query";
import { bookingService } from "@services/bookingService";
import { DataTable } from "@components/tables/DataTable";
import { PageHeader } from "@components/shared/PageHeader";
import { StatusBadge } from "@components/shared/StatusBadge";
import { DateRangePicker } from "@components/shared/DateRangePicker";
import { ExportButton } from "@components/shared/ExportButton";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@components/ui/select";
import { Button } from "@components/ui/button";
import { useRouter } from "next/navigation";
import { formatDateTime, formatCurrency } from "@lib/utils";
import { Eye } from "lucide-react";
import type { ColumnDef, PaginationState, SortingState } from "@tanstack/react-table";
import type { Booking, DateRange } from "@types/admin";

export default function BookingsPage() {
  const router = useRouter();
  const [status, setStatus] = useState("all");
  const [bookingType, setBookingType] = useState("all");
  const [dateRange, setDateRange] = useState<DateRange | undefined>();
  const [pagination, setPagination] =
    useState<PaginationState>({ pageIndex: 0, pageSize: 20 });
  const [sorting, setSorting] = useState<SortingState>([]);

  const { data, isLoading } = useQuery({
    queryKey: ["bookings", { status, bookingType, dateRange, pagination, sorting }],
    queryFn: () =>
      bookingService.listBookings({
        status: status !== "all" ? status : undefined,

```

```

        booking_type: bookingType !== "all" ? bookingType :
undefined,
        date_from: dateRange?.from?.toISOString(),
        date_to: dateRange?.to?.toISOString(),
        page: pagination.pageIndex + 1,
        page_size: pagination.pageSize,
        ordering: sorting[0] ? `${sorting[0].desc ? "-" : ""}${
{sorting[0].id}` : undefined,
    }},
    placeholderData: (prev) => prev,
  });

  const columns: ColumnDef<Booking>[] = [
    { accessorKey: "id", header: "ID", cell: ({ getValue }) => `#${
{getValue()}` },
    { accessorKey: "user_name", header: "User" },
    { accessorKey: "guard_name", header: "Guard" },
    {
      accessorKey: "status",
      header: "Status",
      cell: ({ getValue }) => <StatusBadge
status={getValue<string>()} />,
    },
    { accessorKey: "booking_type", header: "Type" },
    {
      accessorKey: "amount",
      header: "Amount",
      cell: ({ getValue }) => formatCurrency(getValue<number>()),
    },
    {
      accessorKey: "created_at",
      header: "Created",
      cell: ({ getValue }) => formatDateTime(getValue<string>()),
    },
    {
      id: "actions",
      header: "",
      cell: ({ row }) => (
        <Button
          variant="ghost"
          size="sm"
          onClick={() => router.push(`/bookings/${row.original.id}
`)}
        >
        <Eye className="h-4 w-4 mr-1" />
        View
      </Button>
    ),
  ],
];

return (
  <div className="space-y-6">
    <PageHeader

```



```

        title="Bookings"
        description="All platform bookings"
        action={
            <ExportButton endpoint="/admin/bookings/export/"
filename="bookings.csv" />
        }
    />

    <div className="flex flex-wrap gap-3">
        <Select value={status} onValueChange={setStatus}>
            <SelectTrigger className="w-40">
                <SelectValue placeholder="Status" />
            </SelectTrigger>
            <SelectContent>
                <SelectItem value="all">All Statuses</SelectItem>
                <SelectItem value="pending">Pending</SelectItem>
                <SelectItem value="active">Active</SelectItem>
                <SelectItem value="completed">Completed</SelectItem>
                <SelectItem value="cancelled">Cancelled</SelectItem>
            </SelectContent>
        </Select>

        <Select value={bookingType} onValueChange={setBookingType}
>
            <SelectTrigger className="w-40">
                <SelectValue placeholder="Type" />
            </SelectTrigger>
            <SelectContent>
                <SelectItem value="all">All Types</SelectItem>
                <SelectItem value="hourly">Hourly</SelectItem>
                <SelectItem value="daily">Daily</SelectItem>
                <SelectItem value="event">Event</SelectItem>
            </SelectContent>
        </Select>

        <DateRangePicker value={dateRange}
onChange={setDateRange} />
    </div>

    <DataTable
        columns={columns}
        data={data?.results ?? []}
        loading={isLoading}
        totalCount={data?.count ?? 0}
        pagination={pagination}
        onPaginationChange={setPagination}
        sorting={sorting}
        onSortingChange={setSorting}
        pageSize={pagination.pageSize}
    />
</div>
);
}

```

8. Booking Detail

File: app/(dashboard)/bookings/[id]/page.tsx

```
"use client";

import { useState } from "react";
import { useParams } from "next/navigation";
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { bookingService } from "@services/bookingService";
import { PageHeader } from "@components/shared/PageHeader";
import { StatusBadge } from "@components/shared/StatusBadge";
import { BookingTimeline } from "@components/shared/BookingTimeline";
import { ConfirmDialog } from "@components/shared/ConfirmDialog";
import { Button } from "@components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { formatDateTime, formatCurrency } from "@lib/utils";
import { XCircle, RefreshCw } from "lucide-react";
import { toast } from "sonner";

export default function BookingDetailPage() {
  const { id } = useParams<{ id: string }>();
  const qc = useQueryClient();
  const [cancelOpen, setCancelOpen] = useState(false);
  const [refundOpen, setRefundOpen] = useState(false);

  const { data: booking, isLoading } = useQuery({
    queryKey: ["booking", id],
    queryFn: () => bookingService.getBooking(id),
  });

  const cancelMutation = useMutation({
    mutationFn: () => bookingService.forceCancelBooking(id),
    onSuccess: () => {
      toast.success("Booking cancelled");
      qc.invalidateQueries({ queryKey: ["booking", id] });
      setCancelOpen(false);
    },
    onError: (err: Error) => toast.error(err.message),
  });

  const refundMutation = useMutation({
    mutationFn: () => bookingService.refundBooking(id),
    onSuccess: () => {
      toast.success("Refund issued");
      qc.invalidateQueries({ queryKey: ["booking", id] });
      setRefundOpen(false);
    },
    onError: (err: Error) => toast.error(err.message),
  });
}
```

```

    if (isLoading || !booking) return null;

    return (
      <div className="space-y-6">
        <PageHeader
          title={`Booking #${booking.id}`}
          description={formatDateTime(booking.created_at)}
          action={
            <div className="flex gap-2">
              {["pending", "active"].includes(booking.status) && (
                <Button variant="destructive" size="sm" onClick={() => setCancelOpen(true)}>
                  <XCircle className="mr-2 h-4 w-4" /> Force Cancel
                </Button>
              )}
              {booking.status === "completed" && !booking.refunded
                && (
                  <Button variant="outline" size="sm" onClick={() => setRefundOpen(true)}>
                    <RefreshCw className="mr-2 h-4 w-4" /> Issue Refund
                  </Button>
                )}
            </div>
          }
        </div>
      </div>

      <div className="grid grid-cols-1 gap-6 xl:grid-cols-2">
        <Card>
          <CardHeader><CardTitle className="text-base">Booking Info</CardTitle></CardHeader>
          <CardContent>
            <dl className="space-y-3">
              {[
                { label: "Status", value: <StatusBadge status={booking.status} /> },
                { label: "User", value: booking.user_name },
                { label: "Guard", value: booking.guard_name },
                { label: "Type", value: booking.booking_type },
                { label: "Location", value: booking.location_address },
                { label: "Start Time", value: formatDateTime(booking.start_time) },
                { label: "End Time", value: booking.end_time ? formatDateTime(booking.end_time) : "-" },
              ].map(({ label, value }) => (
                <div key={label} className="flex items-start justify-between gap-4">
                  <dt className="text-sm text-gray-500 min-w-[100px]">{label}</dt>
                  <dd className="text-sm font-medium text-right">{value}</dd>
                </div>
              ))}
            </dl>
          </CardContent>
        </Card>
      </div>
    )
  )
}
</div>

```

```

    ))}
  </dl>
</CardContent>
</Card>

<Card>
  <CardHeader><CardTitle className="text-base">Amount
Breakdown</CardTitle></CardHeader>
  <CardContent>
    <dl className="space-y-3">
      {[
        { label: "Base Amount", value:
formatCurrency(booking.base_amount) },
        { label: "Platform Fee", value:
formatCurrency(booking.platform_fee) },
        { label: "Guard Payout", value:
formatCurrency(booking.guard_payout) },
        { label: "Total Charged", value:
formatCurrency(booking.amount), bold: true },
      ].map(({ label, value, bold }) => (
        <div key={label} className={`flex justify-between
${bold ? "border-t pt-3 font-semibold" : ""}`}>
          <dt className="text-sm text-gray-500">{label}</
dt>
          <dd className={`text-sm ${bold ? "font-bold" :
"font-medium"}`}>{value}</dd>
        </div>
      ))}
    </dl>
  </CardContent>
</Card>
</div>

{ /* Status Timeline */}
<Card>
  <CardHeader><CardTitle className="text-base">Status
Timeline</CardTitle></CardHeader>
  <CardContent>
    <BookingTimeline events={booking.status_history ?? []} /
>
  </CardContent>
</Card>

{ /* Tracking Map */}
{booking.tracking_path && (
  <Card>
    <CardHeader><CardTitle className="text-base">Tracking
Path</CardTitle></CardHeader>
    <CardContent className="p-0 rounded-b-lg overflow-
hidden">
      <iframe
        title="Booking tracking"
        width="100%"
        height="300"

```

```

        className="border-0"
        src={`https://www.google.com/maps/embed/v1/view?
key=${process.env.NEXT_PUBLIC_GOOGLE_MAPS_KEY}&center=$
${booking.tracking_path[0]?.lat},${booking.tracking_path[0]?.lng}
&zoom=14`}
      />
    </CardContent>
  </Card>
)}

    <ConfirmDialog open={cancelOpen}
onOpenChange={setCancelOpen} title="Force Cancel Booking"
description="This will cancel the booking and may trigger a
refund. Continue?" confirmLabel="Cancel Booking"
variant="destructive" onConfirm={() => cancelMutation.mutate()}
loading={cancelMutation.isPending} />
    <ConfirmDialog open={refundOpen}
onOpenChange={setRefundOpen} title="Issue Refund"
description="This will refund the full booking amount to the
user's payment method." confirmLabel="Issue Refund"
variant="default" onConfirm={() => refundMutation.mutate()}
loading={refundMutation.isPending} />
  </div>
);
}

```

9. Payments Page

File: app/(dashboard)/payments/page.tsx

```

"use client";

import { useState } from "react";
import { useQuery } from "@tanstack/react-query";
import { paymentService } from "@services/paymentService";
import { DataTable } from "@components/tables/DataTable";
import { PageHeader } from "@components/shared/PageHeader";
import { StatusBadge } from "@components/shared/StatusBadge";
import { KPICard } from "@components/shared/KPICard";
import { DateRangePicker } from "@components/shared/
DateRangePicker";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/
components/ui/tabs";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@components/ui/select";
import { formatDateTime, formatCurrency } from "@lib/utils";

```

```

import { DollarSign, TrendingUp, CreditCard } from "lucide-react";
import type { ColumnDef, PaginationState } from "@tanstack/react-table";
import type { Transaction, DateRange } from "@types/admin";

const transactionCols: ColumnDef<Transaction>[] = [
  { accessorKey: "id", header: "ID", cell: ({ getValue }) => `#${getValue()}` },
  { accessorKey: "reference", header: "Reference" },
  { accessorKey: "type", header: "Type" },
  { accessorKey: "user_name", header: "User" },
  { accessorKey: "amount", header: "Amount", cell: ({ getValue })
=> formatCurrency(getValue<number>()) },
  { accessorKey: "status", header: "Status", cell: ({ getValue })
=> <StatusBadge status={getValue<string>()} /> },
  { accessorKey: "created_at", header: "Date", cell:
({ getValue }) => formatDateTime(getValue<string>()) },
];

export default function PaymentsPage() {
  const [txType, setTxType] = useState("all");
  const [txStatus, setTxStatus] = useState("all");
  const [dateRange, setDateRange] = useState<DateRange |
undefined>();
  const [pagination, setPagination] =
useState<PaginationState>({ pageIndex: 0, pageSize: 20 });

  const { data: summary } = useQuery({
    queryKey: ["payments", "summary"],
    queryFn: paymentService.getRevenueSummary,
    refetchInterval: 60_000,
  });

  const { data: transactions, isLoading } = useQuery({
    queryKey: ["transactions", { txType, txStatus, dateRange,
pagination }],
    queryFn: () =>
      paymentService.listTransactions({
        type: txType !== "all" ? txType : undefined,
        status: txStatus !== "all" ? txStatus : undefined,
        date_from: dateRange?.from?.toISOString(),
        date_to: dateRange?.to?.toISOString(),
        page: pagination.pageIndex + 1,
        page_size: pagination.pageSize,
      }),
    placeholderData: (prev) => prev,
  });

  return (
    <div className="space-y-6">
      <PageHeader title="Payments" description="Transactions,
payouts, and revenue" />

      { /* Revenue summary KPIs */ }
    </div>
  );
}

```

```

        <div className="grid grid-cols-1 gap-4 sm:grid-cols-3">
            <KPICard title="Revenue Today"
value={formatCurrency(summary?.today ?? 0)} icon={DollarSign}
color="green" />
            <KPICard title="Revenue This Month"
value={formatCurrency(summary?.this_month ?? 0)} icon={TrendingUp}
color="blue" />
            <KPICard title="Total Transactions"
value={summary?.total_count ?? 0} icon={CreditCard}
color="purple" />
        </div>

        <Tabs defaultValue="transactions">
            <TabsList>
                <TabsTrigger value="transactions">Transactions</
TabsTrigger>
                <TabsTrigger value="payouts">Payouts</TabsTrigger>
            </TabsList>

            <TabsContent value="transactions" className="space-y-4
pt-4">
                <div className="flex flex-wrap gap-3">
                    <Select value={txType} onChange={setTxType}>
                        <SelectTrigger className="w-36"><SelectValue
placeholder="Type" /></SelectTrigger>
                        <SelectContent>
                            <SelectItem value="all">All Types</SelectItem>
                            <SelectItem value="booking">Booking</SelectItem>
                            <SelectItem value="refund">Refund</SelectItem>
                            <SelectItem value="payout">Payout</SelectItem>
                        </SelectContent>
                    </Select>
                    <Select value={txStatus} onChange={setTxStatus}>
                        <SelectTrigger className="w-36"><SelectValue
placeholder="Status" /></SelectTrigger>
                        <SelectContent>
                            <SelectItem value="all">All</SelectItem>
                            <SelectItem value="success">Success</SelectItem>
                            <SelectItem value="pending">Pending</SelectItem>
                            <SelectItem value="failed">Failed</SelectItem>
                        </SelectContent>
                    </Select>
                    <DatePicker value={dateRange}
onChange={setDateRange} />
                </div>

                <DataTable
                    columns={transactionCols}
                    data={transactions?.results ?? []}
                    loading={isLoading}
                    totalCount={transactions?.count ?? 0}
                    pagination={pagination}
                    onPaginationChange={setPagination}
                    pageSize={pagination.pageSize}

```

```

        />
      </TabsContent>

      <TabsContent value="payouts" className="pt-4">
        <PayoutsTab />
      </TabsContent>
    </Tabs>
  </div>
);
}

// Inline Payouts Tab component
function PayoutsTab() {
  const qc = useQueryClient();
  const [selectedIds, setSelectedIds] = useState<string[]>([]);
  const [pagination, setPagination] =
    useState<PaginationState>({ pageIndex: 0, pageSize: 20 });

  const { data: payouts, isLoading } = useQuery({
    queryKey: ["payouts", "pending", pagination],
    queryFn: () => paymentService.listPayouts({ status: "pending",
    page: pagination.pageIndex + 1, page_size: pagination.pageSize }),
    placeholderData: (prev) => prev,
  });

  const approveMutation = useMutation({
    mutationFn: (ids: string[]) =>
      Promise.all(ids.map((id) =>
        paymentService.approvePayout(id))),
    onSuccess: () => {
      toast.success("Payouts approved");
      setSelectedIds([]);
      qc.invalidateQueries({ queryKey: ["payouts"] });
    },
    onError: (err: Error) => toast.error(err.message),
  });

  const payoutCols: ColumnDef<any>[] = [
    {
      id: "select",
      header: ({ table }) => (
        <input
          type="checkbox"
          checked={table.getIsAllPageRowsSelected()}
          onChange={table.getToggleAllPageRowsSelectedHandler()}
          className="rounded border-gray-300"
        />
      ),
      cell: ({ row }) => (
        <input
          type="checkbox"
          checked={row.getIsSelected()}
          onChange={row.getToggleSelectedHandler()}
          className="rounded border-gray-300"

```



```

        />
      ),
      enableSorting: false,
    },
    { accessorKey: "guard_name", header: "Guard" },
    { accessorKey: "amount", header: "Amount", cell:
({ getValue }) => formatCurrency(getValue<number>()) },
    { accessorKey: "requested_at", header: "Requested", cell:
({ getValue }) => formatDateTime(getValue<string>()) },
    { accessorKey: "bank_name", header: "Bank" },
    { accessorKey: "account_number", header: "Account" },
    {
      id: "actions",
      header: "",
      cell: ({ row }) => (
        <div className="flex gap-2">
          <Button size="sm" className="bg-green-600 hover:bg-
green-700 h-7 text-xs" onClick={() =>
approveMutation.mutate([row.original.id])}>Approve</Button>
          <Button size="sm" variant="destructive" className="h-7
text-xs" onClick={() => rejectMutation.mutate(row.original.id)}
>Reject</Button>
        </div>
      ),
    },
  ],

  const rejectMutation = useMutation({
    mutationFn: (id: string) => paymentService.rejectPayout(id),
    onSuccess: () => {
      toast.success("Payout rejected");
      qc.invalidateQueries({ queryKey: ["payouts"] });
    },
    onError: (err: Error) => toast.error(err.message),
  });

  return (
    <div className="space-y-4">
      {selectedIds.length > 0 && (
        <div className="flex items-center gap-3 rounded-lg bg-
blue-50 border border-blue-200 px-4 py-2">
          <span className="text-sm text-
blue-700">{selectedIds.length} payouts selected</span>
          <Button size="sm" className="bg-green-600 hover:bg-
green-700 h-7 text-xs" onClick={() =>
approveMutation.mutate(selectedIds)}
disabled={approveMutation.isPending}>
            Bulk Approve
          </Button>
        </div>
      )}
    <DataTable
      columns={payoutCols}
      data={payouts?.results ?? []}

```

```

        loading={isLoading}
        totalCount={payouts?.count ?? 0}
        pagination={pagination}
        onPaginationChange={setPagination}
        pageSize={pagination.pageSize}
        onRowSelectionChange={(ids) => setSelectedIds(ids)}
      />
    </div>
  );
}

```

```

import { useMutation } from "@tanstack/react-query";
import { useQueryClient } from "@tanstack/react-query";
import { toast } from "sonner";

```

10. SOS Dashboard

File: app/(dashboard)/sos/page.tsx

```

"use client";

import { useEffect, useRef } from "react";
import { useQuery } from "@tanstack/react-query";
import { sosService } from "@services/sosService";
import { useSosStore } from "@stores/sosStore";
import { useAdminWebSocket } from "@hooks/useAdminWebSocket";
import { SOSEventCard } from "@components/shared/SOSEventCard";
import { DataTable } from "@components/tables/DataTable";
import { PageHeader } from "@components/shared/PageHeader";
import { StatusBadge } from "@components/shared/StatusBadge";
import { formatDateTime } from "@lib/utils";
import { AlertTriangle } from "lucide-react";
import type { ColumnDef } from "@tanstack/react-table";
import type { SOSEvent } from "@types/admin";

const historyColumns: ColumnDef<SOSEvent>[] = [
  { accessorKey: "id", header: "ID", cell: ({ getValue }) => `#${getValue()}` },
  { accessorKey: "user_name", header: "User" },
  { accessorKey: "guard_name", header: "Guard" },
  { accessorKey: "location_address", header: "Location" },
  { accessorKey: "status", header: "Status", cell: ({ getValue })
=> <StatusBadge status={getValue<string>()} /> },
  { accessorKey: "created_at", header: "Triggered", cell:
({ getValue }) => formatDateTime(getValue<string>()) },
  { accessorKey: "resolved_at", header: "Resolved", cell:
({ getValue }) => getValue<string>() ?
formatDateTime(getValue<string>()) : "-" },
];

export default function SOSPage() {

```

```

useAdminWebSocket();

const { activeEvents } = useSosStore();
const alarmRef = useRef<HTMLAudioElement | null>(null);
const previousCount = useRef(0);

const { data: history, isLoading } = useQuery({
  queryKey: ["sos", "history"],
  queryFn: () => sosService.getSOSHistory({ page: 1, page_size:
50 }),
  refetchInterval: 30_000,
});

// Play alarm and show notification on new SOS
useEffect(() => {
  if (activeEvents.length > previousCount.current) {
    // Play alarm
    if (alarmRef.current) {
      alarmRef.current.currentTime = 0;
      alarmRef.current.play().catch(() => {});
    }

    // Browser notification
    if (
      typeof window !== "undefined" &&
      "Notification" in window &&
      Notification.permission === "granted" &&
      document.hidden
    ) {
      new Notification("🚨 New SOS Alert", {
        body: "A new SOS event has been triggered on the
platform.",
        icon: "/favicon.ico",
        tag: "sos-alert",
      });
    }
  }
  previousCount.current = activeEvents.length;
}, [activeEvents.length]);

return (
  <div className="space-y-6">
    <audio ref={alarmRef} src="/sounds/sos-alarm.mp3"
preload="auto" />

    <PageHeader
      title="SOS Dashboard"
      description="Real-time emergency event management"
      action={
        activeEvents.length > 0 ? (
          <div className="flex items-center gap-2 rounded-lg bg-
red-50 border border-red-200 px-4 py-2">
            <AlertTriangle className="h-4 w-4 text-red-600
animate-pulse" />

```

```

        <span className="text-sm font-semibold text-
red-700">
            {activeEvents.length} Active SOS
        </span>
    </div>
    ) : null
}
/>

{/* Active SOS Cards */}
{activeEvents.length === 0 ? (
    <div className="flex flex-col items-center justify-center
rounded-xl border-2 border-dashed border-gray-200 py-16">
        <AlertTriangle className="h-10 w-10 text-gray-300
mb-3" />
        <p className="text-sm text-gray-500 font-medium">No
active SOS events</p>
        <p className="text-xs text-gray-400 mt-1">New events
will appear here in real-time</p>
    </div>
    ) : (
        <div className="grid grid-cols-1 gap-4 sm:grid-cols-2
xl:grid-cols-3">
            {activeEvents.map((event) => (
                <SOSEventCard key={event.id} event={event} />
            ))}
        </div>
    )}

{/* Historical SOS */}
<div>
    <h2 className="text-base font-semibold mb-4">SOS History</
h2>
    <DataTable
        columns={historyColumns}
        data={history?.results ?? []}
        loading={isLoading}
        totalCount={history?.count ?? 0}
        pageSize={20}
    />
</div>
</div>
);
}

```

11. Analytics Page

File: app/(dashboard)/analytics/page.tsx

"use client";

```

import { useState } from "react";
import { useQuery } from "@tanstack/react-query";
import { analyticsService } from "@services/analyticsService";
import { PageHeader } from "@components/shared/PageHeader";
import { DateRangePicker } from "@components/shared/
DateRangePicker";
import { Card, CardContent, CardHeader, CardTitle } from "@/
components/ui/card";
import {
  LineChart,
  Line,
  BarChart,
  Bar,
  PieChart,
  Pie,
  Cell,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  Legend,
  ResponsiveContainer,
} from "recharts";
import { formatCurrency } from "@lib/utils";
import type { DateRange } from "@types/admin";

const TIER_COLORS = {
  bronze: "#cd7f32",
  silver: "#c0c0c0",
  gold: "#ffd700",
  platinum: "#e5e4e2",
};

const STATUS_COLORS = {
  completed: "#22c55e",
  cancelled: "#ef4444",
  active: "#3b82f6",
  pending: "#f59e0b",
};

export default function AnalyticsPage() {
  const [dateRange, setDateRange] = useState<DateRange |
undefined>({
    from: new Date(Date.now() - 30 * 24 * 60 * 60 * 1000),
    to: new Date(),
  });

  const params = {
    date_from: dateRange?.from?.toISOString(),
    date_to: dateRange?.to?.toISOString(),
  };

  const { data: bookingAnalytics } = useQuery({
    queryKey: ["analytics", "bookings", params],
  });

```

```

    queryFn: () => analyticsService.getBookingAnalytics(params),
  });

  const { data: revenueAnalytics } = useQuery({
    queryKey: ["analytics", "revenue", params],
    queryFn: () => analyticsService.getRevenueAnalytics(params),
  });

  const { data: guardAnalytics } = useQuery({
    queryKey: ["analytics", "guards", params],
    queryFn: () => analyticsService.getGuardAnalytics(params),
  });

  return (
    <div className="space-y-6">
      <PageHeader
        title="Analytics"
        description="Platform performance metrics and trends"
        action={<DateRangePicker value={dateRange}
onChange={setDateRange} />}
      />

      {/* Daily Bookings Line Chart */}
      <Card>
        <CardHeader>
          <CardTitle className="text-base">Daily Bookings</
CardTitle>
        </CardHeader>
        <CardContent>
          <ResponsiveContainer width="100%" height={300}>
            <LineChart data={bookingAnalytics?.daily ?? []}>
              <CartesianGrid strokeDasharray="3 3"
stroke="#f0f0f0" />
              <XAxis dataKey="date" tick={{ fontSize: 12 }} />
              <YAxis tick={{ fontSize: 12 }} />
              <Tooltip />
              <Legend />
              <Line type="monotone" dataKey="count"
stroke="#111827" strokeWidth={2} dot={false} name="Bookings" />
            </LineChart>
          </ResponsiveContainer>
        </CardContent>
      </Card>

      {/* Revenue Bar Chart */}
      <Card>
        <CardHeader>
          <CardTitle className="text-base">Revenue by Day</
CardTitle>
        </CardHeader>
        <CardContent>
          <ResponsiveContainer width="100%" height={300}>
            <BarChart data={revenueAnalytics?.daily ?? []}>
              <CartesianGrid strokeDasharray="3 3"

```

```

stroke="#f0f0f0" />
      <XAxis dataKey="date" tick={{ fontSize: 12 }} />
      <YAxis tick={{ fontSize: 12 }} tickFormatter={(v) =>
`R${v}`} />
      <Tooltip formatter={(v: number) =>
formatCurrency(v)} />
      <Bar dataKey="revenue" fill="#111827" radius={[4, 4,
0, 0]} name="Revenue" />
    </BarChart>
  </ResponsiveContainer>
</CardContent>
</Card>

<div className="grid grid-cols-1 gap-6 xl:grid-cols-2">
  { /* Guard Tier Distribution Pie Chart */ }
  <Card>
    <CardHeader>
      <CardTitle className="text-base">Guard Tier
Distribution</CardTitle>
    </CardHeader>
    <CardContent>
      <ResponsiveContainer width="100%" height={280}>
        <PieChart>
          <Pie
            data={guardAnalytics?.tier_distribution ?? []}
            dataKey="count"
            nameKey="tier"
            cx="50%"
            cy="50%"
            outerRadius={100}
            label={({ tier, percent }) =>
              `${tier} ${(percent * 100).toFixed(0)}%`
            }
          />
        </Pie>
        {
          (guardAnalytics?.tier_distribution ??
[ ]).map((entry: any) => (
            <Cell
              key={entry.tier}
              fill={TIER_COLORS[entry.tier as keyof typeof
TIER_COLORS] ?? "#6b7280"}
            />
          ))
        }
      </PieChart>
      <Tooltip />
    </PieChart>
  </ResponsiveContainer>
</CardContent>
</Card>

  { /* Booking Status Donut */ }
  <Card>
    <CardHeader>
      <CardTitle className="text-base">Booking Status
Breakdown</CardTitle>

```

```

</CardHeader>
<CardContent>
  <ResponsiveContainer width="100%" height={280}>
    <PieChart>
      <Pie
        data={bookingAnalytics?.status_breakdown ?? []}
        dataKey="count"
        nameKey="status"
        cx="50%"
        cy="50%"
        innerRadius={70}
        outerRadius={100}
        label={({ status, percent }) =>
          `${status} ${(percent * 100).toFixed(0)}%`
        }
      />
    >
      {(bookingAnalytics?.status_breakdown ??
[]).map((entry: any) => (
        <Cell
          key={entry.status}
          fill={STATUS_COLORS[entry.status as keyof
typeof STATUS_COLORS] ?? "#6b7280"}
        />
      ))}
    </Pie>
    <Tooltip />
    <Legend />
  </PieChart>
</ResponsiveContainer>
</CardContent>
</Card>
</div>

```

```

{/* Top 10 Guards by Revenue – Horizontal Bar */}
<Card>
  <CardHeader>
    <CardTitle className="text-base">Top 10 Guards by
Revenue</CardTitle>
  </CardHeader>
  <CardContent>
    <ResponsiveContainer width="100%" height={350}>
      <BarChart
        data={guardAnalytics?.top_guards ?? []}
        layout="vertical"
        margin={{ left: 100 }}
      />
      <CartesianGrid strokeDasharray="3 3"
stroke="#f0f0f0" />
      <XAxis type="number" tickFormatter={(v) => `R${v}`}>
        tick={{ fontSize: 12 }} />
      <YAxis type="category" dataKey="name">
        tick={{ fontSize: 12 }} width={100} />
      <Tooltip formatter={(v: number) =>
formatCurrency(v)} />
    </ResponsiveContainer>
  </CardContent>
</Card>

```



```

        <Bar dataKey="revenue" fill="#111827" radius={[0, 4,
4, 0]} name="Revenue" />
      </BarChart>
    </ResponsiveContainer>
  </CardContent>
</Card>

  {/* Peak Hours Heatmap */}
  <Card>
    <CardHeader>
      <CardTitle className="text-base">Peak Hours Heatmap</
CardTitle>
    </CardHeader>
    <CardContent>
      <PeakHoursHeatmapGrid
data={bookingAnalytics?.peak_hours ?? []} />
    </CardContent>
  </Card>
</div>
);
}

// Heatmap grid component
function PeakHoursHeatmapGrid({ data }: { data: Array<{ day:
number; hour: number; count: number }> }) {
  const days = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
  const hours = Array.from({ length: 24 }, (_, i) => i);
  const maxCount = Math.max(...data.map((d) => d.count), 1);

  const getCount = (day: number, hour: number) =>
    data.find((d) => d.day === day && d.hour === hour)?.count ??
0;

  return (
    <div className="overflow-x-auto">
      <div className="inline-block min-w-full">
        <div className="flex gap-1 mb-1">
          <div className="w-10" />
          {hours.map((h) => (
            <div key={h} className="w-8 text-center text-xs text-
gray-400">
              {h}
            </div>
          ))}
        </div>
        {days.map((day, dayIdx) => (
          <div key={day} className="flex gap-1 mb-1">
            <div className="w-10 text-xs text-gray-500 flex items-
center">{day}</div>
            {hours.map((hour) => {
              const count = getCount(dayIdx, hour);
              const intensity = count / maxCount;
              return (
                <div

```

```

        key={hour}
        className="w-8 h-8 rounded-sm cursor-default"
        style={{
            backgroundColor: `rgba(17, 24, 39, $
{intensity})`,
        }}
        title={` ${day} ${hour}:00 - ${count} bookings`}
    />
    );
    })}
</div>
)))
<div className="flex items-center gap-2 mt-3 text-xs text-
gray-400">
    <span>Less</span>
    {[0.1, 0.3, 0.5, 0.7, 0.9].map((i) => (
        <div
            key={i}
            className="w-5 h-5 rounded-sm"
            style={{ backgroundColor: `rgba(17, 24, 39, $
{i})` }}
        />
    ))}
    <span>More</span>
</div>
</div>
</div>
);
}

```

12. Verifications Page

File: app/(dashboard)/verifications/page.tsx

```

"use client";

import { useQuery } from "@tanstack/react-query";
import { guardService } from "@services/guardService";
import { DataTable } from "@components/tables/DataTable";
import { PageHeader } from "@components/shared/PageHeader";
import { KPICard } from "@components/shared/KPICard";
import { Button } from "@components/ui/button";
import { useRouter } from "next/navigation";
import { formatDate } from "@lib/utils";
import { BadgeCheck, Clock, XCircle, Eye } from "lucide-react";
import type { ColumnDef, PaginationState } from "@tanstack/react-
table";
import { useState } from "react";

interface VerificationQueueItem {
    id: string;

```

```

guard_name: string;
guard_photo: string;
submitted_at: string;
document_count: number;
days_waiting: number;
}

export default function VerificationsPage() {
  const router = useRouter();
  const [pagination, setPagination] =
    useState<PaginationState>({ pageIndex: 0, pageSize: 20 });

  const { data: queue, isLoading } = useQuery({
    queryKey: ["verifications", "queue", pagination],
    queryFn: () =>
      guardService.getVerificationQueue({
        page: pagination.pageIndex + 1,
        page_size: pagination.pageSize,
      }),
    refetchInterval: 30_000,
  });

  const { data: stats } = useQuery({
    queryKey: ["verifications", "stats"],
    queryFn: guardService.getVerificationStats,
    refetchInterval: 30_000,
  });

  const columns: ColumnDef<VerificationQueueItem>[] = [
    { accessorKey: "guard_name", header: "Guard Name" },
    {
      accessorKey: "submitted_at",
      header: "Submitted",
      cell: ({ getValue }) => formatDate(getValue<string>()),
    },
    { accessorKey: "document_count", header: "Documents" },
    {
      accessorKey: "days_waiting",
      header: "Waiting",
      cell: ({ getValue }) => {
        const days = getValue<number>();
        return (
          <span className={days > 3 ? "text-red-600 font-
semibold" : "text-gray-900"}>
            {days}d
          </span>
        );
      },
    },
    {
      id: "actions",
      header: "",
      cell: ({ row }) => (
        <Button

```

```

        variant="ghost"
        size="sm"
        onClick={() => router.push(`/guards/${row.original.id}`)}
    `)}
    >
    <Eye className="h-4 w-4 mr-1" />
    Review
  </Button>
),
},
];

return (
  <div className="space-y-6">
    <PageHeader
      title="Verifications"
      description="Guard document verification queue"
    />

    { /* KPIs */ }
    <div className="grid grid-cols-1 gap-4 sm:grid-cols-3">
      <KPICard
        title="Pending Review"
        value={stats?.pending ?? 0}
        icon={Clock}
        color="yellow"
      />
      <KPICard
        title="Approved Today"
        value={stats?.approved_today ?? 0}
        icon={BadgeCheck}
        color="green"
      />
      <KPICard
        title="Rejected Today"
        value={stats?.rejected_today ?? 0}
        icon={XCircle}
        color="red"
      />
    </div>

    <DataTable
      columns={columns}
      data={queue?.results ?? []}
      loading={isLoading}
      totalCount={queue?.count ?? 0}
      pagination={pagination}
      onPageChange={setPagination}
      pageSize={pagination.pageSize}
      onClick={row => router.push(`/guards/${row.id}`)}
    />
  </div>
);
}

```

13. Settings Page

File: app/(dashboard)/settings/page.tsx

```
"use client";

import { useEffect } from "react";
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { settingsService } from "@services/settingsService";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";
import { PageHeader } from "@components/shared/PageHeader";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
import { Loader2, Save } from "lucide-react";
import { toast } from "sonner";

const settingsSchema = z.object({
  platform_fee_percent: z
    .number({ invalid_type_error: "Must be a number" })
    .min(0, "Must be >= 0")
    .max(100, "Must be <= 100"),
  minimum_payout_threshold: z
    .number({ invalid_type_error: "Must be a number" })
    .min(0, "Must be >= 0"),
  tier_bronze_hourly_rate: z.number().min(0),
  tier_silver_hourly_rate: z.number().min(0),
  tier_gold_hourly_rate: z.number().min(0),
  tier_platinum_hourly_rate: z.number().min(0),
});

type SettingsForm = z.infer<typeof settingsSchema>;

export default function SettingsPage() {
  const qc = useQueryClient();

  const { data: settings, isLoading } = useQuery({
    queryKey: ["settings"],
    queryFn: settingsService.getSettings,
  });

  const {
    register,
    handleSubmit,
    reset,
    formState: { errors, isDirty, isSubmitting },
  }
```

```

} = useForm<SettingsForm>({
  resolver: zodResolver(settingsSchema),
});

useEffect(() => {
  if (settings) reset(settings);
}, [settings, reset]);

const updateMutation = useMutation({
  mutationFn: settingsService.updateSettings,
  onSuccess: () => {
    toast.success("Settings saved successfully");
    qc.invalidateQueries({ queryKey: ["settings"] });
  },
  onError: (err: Error) => toast.error(err.message),
});

if (isLoading) {
  return (
    <div className="flex items-center justify-center h-64">
      <Loader2 className="h-8 w-8 animate-spin text-gray-400" />
    </div>
  );
}

return (
  <div className="space-y-6 max-w-2xl">
    <PageHeader title="Settings" description="Platform-wide configuration" />

    <form onSubmit={handleSubmit((data) =>
updateMutation.mutate(data))} className="space-y-6">
      <\/* Financial Settings */>
      <Card>
        <CardHeader>
          <CardTitle className="text-base">Financial Settings<\/
CardTitle>
          <CardDescription>
            Configure platform fees and payout thresholds
          <\/CardDescription>
        <\/CardHeader>
        <CardContent className="space-y-4">
          <div className="space-y-1.5">
            <Label htmlFor="platform_fee_percent">Platform Fee
(%)<\/Label>
            <Input
              id="platform_fee_percent"
              type="number"
              step="0.1"
              {...register("platform_fee_percent",
{ valueAsNumber: true })}>
              <\/>
            <errors.platform_fee_percent && (
              <p className="text-xs text-red-600">

```

```

        {errors.platform_fee_percent.message}
      </p>
    )}
  </div>

  <div className="space-y-1.5">
    <Label htmlFor="minimum_payout_threshold">
      Minimum Payout Threshold (ZAR)
    </Label>
    <Input
      id="minimum_payout_threshold"
      type="number"
      step="1"
      {...register("minimum_payout_threshold",
{ valueAsNumber: true })}
    />
    {errors.minimum_payout_threshold && (
      <p className="text-xs text-red-600">
        {errors.minimum_payout_threshold.message}
      </p>
    )}
  </div>
</CardContent>
</Card>

{/* Guard Tier Hourly Rates */}
<Card>
  <CardHeader>
    <CardTitle className="text-base">Guard Tier Hourly
Rates</CardTitle>
    <CardDescription>
      Set the hourly rate for each guard tier (ZAR)
    </CardDescription>
  </CardHeader>
  <CardContent className="grid grid-cols-2 gap-4">
    {[["bronze", "silver", "gold", "platinum"] as
const).map((tier) => (
      <div key={tier} className="space-y-1.5">
        <Label
          htmlFor={`tier_${tier}_hourly_rate`}
          className="capitalize"
        >
          {tier}
        </Label>
        <Input
          id={`tier_${tier}_hourly_rate`}
          type="number"
          step="1"
          {...register(`tier_${tier}_hourly_rate` as keyof
SettingsForm, {
            valueAsNumber: true,
          })}
        />
        {errors[`tier_${tier}_hourly_rate` as keyof typeof

```

```

errors] && (
    <p className="text-xs text-red-600">
        {errors[`tier_${tier}_hourly_rate` as keyof
typeof errors]?.message}
    </p>
    )}
  </div>
  )})
</CardContent>
</Card>

<div className="flex justify-end">
  <Button
    type="submit"
    disabled={!isDirty || isSubmitting ||
updateMutation.isPending}
    className="bg-gray-900 hover:bg-gray-800"
  >
    {updateMutation.isPending ? (
      <>
        <Loader2 className="mr-2 h-4 w-4 animate-spin" />
        Saving...
      </>
    ) : (
      <>
        <Save className="mr-2 h-4 w-4" />
        Save Settings
      </>
    )}
  </Button>
</div>
</form>
</div>
);
}

```